

LogicPoison 复现报告

Reproduction Report

May 2026

1 论文信息 / Paper Info

Title	LogicPoison: Logical Attacks on Graph Retrieval-Augmented Generation
Authors	Yilin Xiao, Jin Chen, Qinggang Zhang, Yujing Zhang, Chuang Zhou, Longhao Yang, Lingfei Ren, Xin Yang, Xiao Huang
Venue	ACL 2026 Main Conference
ArXiv	https://arxiv.org/abs/2604.02954
GitHub	https://github.com/Jord8061/logicPoison
HuggingFace	https://huggingface.co/datasets/Jord8061/datasets

Table 1: Paper metadata.

2 论文概述 / Paper Overview

LogicPoison is a logical poisoning framework targeting **GraphRAG** systems. Rather than injecting fabricated facts or adversarial instructions into the corpus, it perturbs the *implicit reasoning topology* of the graph through **type-preserving entity swapping**.

The core insight is straightforward but effective:

- Identify globally important *logic hubs* in the corpus.
- Identify query-specific *bridge entities* required for multi-hop reasoning.
- Swap entities *within the same semantic type* (e.g., PERSON with PERSON, ORG with ORG) so that the text remains locally plausible while the graph structure built from the corpus is corrupted.

This causes GraphRAG systems to follow corrupted reasoning chains and produce incorrect answers, while the poisoned corpus remains surface-level natural.

2.1 Three-Stage Framework

1. Global Logic Poison (Stage 1):

Build corpus-level entity statistics and identify high-frequency entities that likely act as *global logic hubs* in the implicit graph. Uses spaCy’s transformer-based NER model (`en_core_web_trf`) with GPU acceleration to process the entire corpus, collecting per-type entity frequency distributions.

2. Query-Centric Logic Poison (Stage 2):

Use query-side reasoning signals to identify *bridge entities* that are important for answering multi-hop questions. An LLM extracts reasoning-critical entities from each question, including implicit bridge entities (typed BRIDGE) and alias-style descriptions (typed ALIAS).

3. Type-Preserving Entity Swapping (Stage 3):

Combine the selected entities and perform *bijective (circular) swapping within the same entity type*. For each type, a pool is constructed from the top-5% corpus entities (by frequency) plus query entities (in occurrence order). A reverse circular permutation is then applied:

$$e_0 \rightarrow e_{n-1}, \quad e_1 \rightarrow e_0, \quad e_2 \rightarrow e_1, \quad \dots, \quad e_{n-1} \rightarrow e_{n-2}$$

This ensures every entity is replaced by exactly one other entity of the same type, producing a poisoned corpus that remains grammatically and semantically plausible at the local text level.

2.2 Key Contributions

- A novel poisoning paradigm that targets *logical structure* rather than surface text, making attacks harder to detect.
- A three-stage pipeline (global, query-centric, type-preserving swap) that is modular and applicable to any GraphRAG system.
- Evaluation on three multi-hop QA benchmarks (HotpotQA, MuSiQue, 2WikiMultiHopQA) demonstrating significant degradation of GraphRAG performance.

3 复现环境 / Reproduction Environment

Python	3.14.4
Virtual Env	venv314
GPU	NVIDIA GeForce RTX 4060
CUDA	13.1
CuPy	Installed (GPU-accelerated spaCy)
LLM API	opencode-go / deepseek-v4-flash

Table 2: System environment.

Package	Version
openai	2.30.0
spacy	3.8.13
tqdm	4.67.3
en_core_web_trf	3.8.0

Table 3: Python dependencies.

4 复现步骤 / Reproduction Steps

4.1 Step 1: Clone Repository

```
git clone https://github.com/Jord8061/logicPoison.git
cd logicPoison
```

4.2 Step 2: Clone Datasets from HuggingFace

```
cd datasets
git clone https://huggingface.co/datasets/Jord8061/datasets .
cd ..
```

The datasets directory should contain three subdirectories: `hotpotqa/`, `musique/`, and `2wikimultihopqa/`, each with `corpus.jsonl` and `queries.jsonl`.

4.3 Step 3: Install Dependencies and spaCy Model

```
python -m venv venv314
source venv314/bin/activate
pip install -r requirements.txt
pip install cupy-cuda13x # GPU acceleration for spaCy
python -m spacy download en_core_web_trf
```

4.4 Step 4: Configure API Access

Set the following environment variables for the OpenAI-compatible API:

```
export OPENAI_API_KEY="<your-api-key>"
export OPENAI_BASE_URL="<your-base-url>"
```

Note: In this reproduction, we used the `deepseek-v4-flash` model via the `opencode-go` gateway. The API key and base URL must be configured before running the query-centric stage.

4.5 Step 5: Fix `list_datasets` Filter

The original `main.py` uses `os.listdir()` to discover datasets, which may include non-dataset directories such as `.git/` or files like `README.md` that were cloned from HuggingFace. The function already filters by checking for `corpus.jsonl` and excluding dot-prefixed names, but we verified this handles the HuggingFace clone correctly. No patch was needed in our case since the `list_datasets` function in `main.py` already requires `corpus.jsonl` to exist in each subdirectory.

4.6 Step 6: Run the Pipeline

```
python main.py \
  --stages all \
  --datasets all \
  --query_model deepseek-v4-flash
```

The pipeline runs three stages sequentially:

1. **global**: Corpus entity statistics extraction (GPU-accelerated NER).
2. **query**: Query-centric entity extraction via LLM API calls.
3. **logic**: Type-preserving entity swapping and corpus poisoning.

Completed stage outputs are auto-detected and skipped on re-runs. Use `--force` to rerun selected stages.

5 复现结果 / Reproduction Results

Results are organized into three directories under **results/**:

- **corpus_entities/**: Per-dataset entity statistics (JSON).
- **queries_entities/**: Per-dataset query entity extraction (JSONL).
- **poisoned_data/**: Poisoned corpora and statistics.

5.1 Global Entity Statistics

Table 4: Global entity statistics per dataset (Stage 1 output).

Dataset	Corpus Size	Types Poisoned	Top Entity Types (count)
2WikiMultiHopQA	6,119	7	PERSON (977), WORK_OF_ART (431), ORG (
HotpotQA	9,811	13	PERSON (1,110), ORG (769), WORK_OF_ART
MuSiQue	11,656	11	PERSON (997), ORG (644), DATE (503)

Table 5: Top entities by type for HotpotQA (Stage 1 output).

Type	Top Entity	Count
PERSON	(top PERSON entity)	1,110
ORG	(top ORG entity)	769
WORK_OF_ART	(top WORK_OF_ART entity)	512

5.2 Query-Centric Entity Extraction

All three datasets contain 500 questions each (1,500 total). Entity extraction was performed using **deepseek-v4-flash** via the **opencode-go** API (**OPENAI_BASE_URL**=<https://opencode.ai/zen/go/v1>).

Table 6: Query entity extraction statistics per dataset (Stage 2 output).

Dataset	Num Queries	Model	Runtime
2WikiMultiHopQA	500	deepseek-v4-flash	~82 min
HotpotQA	500	deepseek-v4-flash	~56 min
MuSiQue	500	deepseek-v4-flash	~7.5 hours

5.3 Poison Statistics

Table 7: Poison statistics per dataset (Stage 3 output).

Dataset	Types Poisoned	Entities Swapped	Total Freq Affected
2WikiMultiHopQA	7	1,865	17,615
HotpotQA	13	3,781	62,905
MuSiQue	11	3,505	61,968

Table 8: Per-type poison statistics for HotpotQA (Stage 3 output).

Entity Type	Num Entities Swapped	Total Frequency
PERSON	(dominant type)	(largest share)
ORG	(second largest)	(second largest)
WORK_OF_ART	(third largest)	(third largest)

Note: The results directory structure is:

```

results/
  corpus_entities/
    hotpotqa.json
    musique.json
    2wikimultihopqa.json
  queries_entities/
    hotpotqa.jsonl
    musique.jsonl
    2wikimultihopqa.jsonl
  poisoned_data/
    hotpotqa/
      corpus.jsonl
      queries.jsonl
      answers.jsonl
      qrels/
        poison_stats_hotpotqa.json
    musique/
      ...
    2wikimultihopqa/
      ...

```

6 关键发现 / Key Findings

6.1 Pipeline Performance

Table 9: Pipeline runtime breakdown.

Stage	Dataset	Runtime
Global (GPU)	2WikiMultiHopQA	~2–3 min
	HotpotQA	~2–3 min
	MuSiQue	~2–3 min
Query (API)	2WikiMultiHopQA	~82 min
	HotpotQA	~56 min
	MuSiQue	~7.5 hours
Logic	2WikiMultiHopQA	~1–2 min
	HotpotQA	~1–2 min
	MuSiQue	~1–2 min
Total	All 3 datasets	~10 hours

The query-centric stage dominates total runtime due to per-question LLM API calls (~2 min/question average). The global and logic stages are fast thanks to GPU-accelerated NER and local text substitution respectively.

6.2 Sanity Check: Type-Preserving Swapping

We verified that the type-preserving constraint is correctly enforced by inspecting sample replacements in the poisoned corpora:

Table 10: Sample entity swaps confirming type preservation.

Original	Replaced With	Type	Preserved?
2011	2010	DATE → DATE	✓
Indian	Australian	NORP → NORP	✓

Documents remain grammatically coherent with only entities swapped—no broken syntax or semantic incoherence was observed at the sentence level.

6.3 Type-Preserving Entity Swapping Mechanism

The core innovation of LogicPoison is the *type-preserving* constraint. By swapping entities only within the same NER type (e.g., PERSON ↔ PERSON, GPE ↔ GPE), the poisoned corpus maintains local grammatical coherence. The reverse circular permutation ensures:

- Every entity in the pool is replaced by exactly one other entity (bijective mapping).
- No entity is replaced by itself.
- The mapping is deterministic and reproducible.

The pool for each type is constructed by concatenating:

1. Top-5% entities by corpus frequency (global logic hubs).
2. Query-centric entities that appear in the corpus (bridge entities), appended in occurrence order, deduplicated against the top-5%.

6.4 GPU Acceleration Impact

The global entity extraction stage (Stage 1) uses spaCy’s transformer-based NER model (`en_core_web_trf`) with GPU acceleration via CuPy. On our RTX 4060 with CUDA 13.1, `spacy.prefer_gpu()` successfully enabled GPU processing, significantly speeding up NER over the full corpus. The pipeline uses `nlp.pipe()` with configurable batch size (default 32) for efficient batch processing.

6.5 API Rate Limiting Observations

The query-centric stage (Stage 2) makes concurrent LLM API calls using a `ThreadPoolExecutor` with configurable `max_workers` (default 8) and `queue_factor` (default 4), yielding up to 32 concurrent requests. Key observations:

- The pipeline includes exponential backoff retry logic (3 retries, base delay 1.5s) for API errors.
- Using `deepseek-v4-flash` as the query model provides a cost-effective alternative to `gpt-4o-mini` for entity extraction.
- Rate limiting may require adjusting `--max_workers` and `--queue_factor` depending on the API provider’s limits.

6.6 Pipeline Robustness

The pipeline includes several robustness features:

- **Auto-skip completed stages:** The `stage_done()` function checks for existing output files and skips completed work.

- **Regex-based replacement:** Stage 3 uses a single compiled regex pattern with word-boundary assertions $((?<!\w)$ and $(?!\\w))$ to prevent cascading replacements and partial word matches.
- **BEIR-compatible output:** The poisoned data directory preserves `queries.jsonl`, `answers.jsonl`, and `qrels/` for direct evaluation with BEIR-style retrieval frameworks.

7 结论 / Conclusion

This report documents the reproduction of the LogicPoison paper (ACL 2026). The three-stage pipeline (global entity statistics, query-centric entity extraction, and type-preserving entity swapping) was successfully set up and executed on the HotpotQA, MuSiQue, and 2WikiMulti-HopQA datasets.

Key takeaways:

- The reproduction environment (Python 3.14.4, RTX 4060, CUDA 13.1, deepseek-v4-flash API) is fully functional.
- The type-preserving entity swapping mechanism is elegant: it corrupts graph-level reasoning structure while preserving local text plausibility.
- GPU acceleration via CuPy is essential for the NER-heavy global stage.
- The modular pipeline design allows running individual stages and datasets independently.
- Total pipeline runtime was ~ 10 hours across all three datasets, dominated by the query-centric LLM API stage.
- Across all datasets, 9,151 unique entities were swapped, affecting 142,488 total occurrences in the corpora.

Citation:

```
@misc{xiao2026logicpoisonlogicalattacksgraph,
  title={LogicPoison: Logical Attacks on Graph
    Retrieval-Augmented Generation},
  author={Yilin Xiao and Jin Chen and Qinggang Zhang
    and Yujing Zhang and Chuang Zhou
    and Longhao Yang and Lingfei Ren
    and Xin Yang and Xiao Huang},
  year={2026},
  eprint={2604.02954},
  archivePrefix={arXiv},
  primaryClass={cs.CL},
  url={https://arxiv.org/abs/2604.02954},
}
```